UHF_READER09.DLL Dynamic Link Library

User's Guide V1.0

1. OPERATION SYSTEM REQUIREMENT:.....1

2. FUNCTION LIST:.....1

2.1) General Function:.....1

2.2) EPCC1-G2 Function:.....1

2.3) 18000-6B Function : 2

3. FUNCTION EXPLANATION:.....3

3.1) General Function:.....3

3.1.1) AutoOpenComPort():.....3

3.1.2) OpenComPort():.....3

3.1.3) CloseComPort():.....4

3.1.4) CloseSpecComPort():.....5

3.1.5)GetReaderInformation():.....5

3.1.6) WriteComAdr ():.....5

3.1.7) WriteScanTime ():.....6

3.1.8) SetPowerDbm ():.....6

3.1.9) Writedfre ():.....6

3.1.10) Writebaud ():.....7

3.2) EPCC1-G2 Function:.....7

3.2.1) Inventory_G2 ():.....7

3.2.2) ReadCard_G2 ():.....8

3.2.3) WriteCard_G2 ():.....8

3.2.4) EraseCard_G2 ():.....9

3.2.5) SetCardProtect_G2 ():.....10

3.2.6) DestroyCard_G2 ():.....11

3.2.7) WriteEPC_G2 ():.....11

3.2.8) SetReadProtect_G2 ():.....11

3.2.9) SetMultiReadProtect_G2 ():.....12

3.2.10) RemoveReadProtect_G2 ():.....12

3.2.11) CheckReadProtected_G2 ():.....13

3.2.12) SetEASAlarm_G2 ():.....13

3.2.13) CheckEASAlarm_G2 ():.....14

3.2.14) LockUserBlock_G2 ():.....14

3.3) 18000-6B Function:.....15

3.3.1) Inventory_6B ():.....15

3.3.2) Inventory2_6B ():.....15

3.3.3) ReadCard_6B ():.....16

3.3.4) WriteCard_6B ():.....16

3.3.5) CheckLock_6B ():.....16

3.3.6) LockByte_6B ():.....17

4. RETURN VALUE DEFINITION.....18

5.ErrorCode Definition.....19

UHF_READER09.DLL is a dynamic link library designed to facilitate EPCC1-G2 and 18000-6B protocol UHF tag application software development.

1. Operation System Requirement:

WINDOWS 2000/XP

2. Function List:

UHF_READER09.DLL includes the following functions：

2.1) General Function:

1) int[] OpenComPort(int[]arr);

2) int[] AutoOpenComPort(int[]arr);

3) int CloseComPort();

4) int CloseSpecComPort(int Frmhandle);

5) int[] GetReaderInformation(int[]arr);

6) int WriteComAdr(int[] arr);

7) int SetPowerDbm(int[] arr);

8) int Writedfre(int[] arr);

9) int Writebaud(int[] arr);

10) int WriteScanTime(int[] arr);

2.2) EPCC1-G2 Function:

1) int[] Inventory_G2(int[]arr);

2) int[] ReadCard_G2(int[]arr);

3) int[] WriteCard_G2(int[]arr);

4) int[] EraseCard_G2(int[]arr);

5) int[] SetCardProtect_G2(int[]arr);

6) int[] DestroyCard_G2(int[]arr);

7) int[] WriteEPC_G2(int[]arr);

8) int[] SetReadProtect_G2(int[]arr);

9) int[] SetMultiReadProtect_G2(int[]arr);

10) int[] RemoveReadProtect_G2(int[]arr);

11) int[] CheckReadProtected_G2(int[]arr);

12) int[] SetEASAlarm_G2(int[]arr);

13) int[] CheckEASAlarm_G2(int[]arr);

14) int[] LockUserBlock_G2(int[]arr);

2.3) 18000-6B Function :

1) int[] Inventory_6B(int[]arr);

2) int[] inventory2_6B(int[]arr);

3) int[] ReadCard_6B(int[]arr);

4) int[] WriteCard_6B(int[]arr);

5) int[] LockByte_6B(int[]arr);

6) int[] CheckLock_6B(int[]arr);

3. Function Explanation:

3.1) General Function:

3.1.1) AutoOpenComPort():

Function description:

This function is used to automatically detect the communication port unoccupied by other application and attached with a reader. The function try to establish the connection between them. The protocol parameters are 57600bps, 8 data bits, 1 start bit, 1 stop bit, no parity bit.

If the connection is established successfully, the function will open the communication port and return a valid handle, otherwise the function will return an error code with a invalid handle(value as -1).

Usage:

int[] AutoOpenComPort(int[]arr);

Parameter:

arr: Input array.two bytes.

First byte:the address of the reader.

When using broadcasting address 0xFF as ComAdr to call the function, the port number to which the reader is detected and the address of the reader will be writed back to parameter Port and ComAdr;

When using a designated address 0x00~0xFE as ComAdr to call the function, the port number to which the reader with the specified address is detected will be writed back to parameter Port.

second：This value set the baud rate of the serial communication control.

baudrate	Actual baud rate
0	9600bps
1	19200 bps
2	38400 bps
4	56000 bps
5	57600 bps
6	115200 bps

Returns:

Return an array when successfully, non-zero value with first byte when error occurred.

Second byte is port(COM1-COM9),third byte is reader address,and last byte is a handle of reader.

3.1.2) OpenComPort():

Function description:

This function is used to establish the connection between the reader and a specified communication port. The protocol parameters are 19200bps, 8 data bits, 1 start bit, 1 stop bit, no parity bit.

Usage:

int[] OpenComPort(int[]arr);

Parameter:

arr: Input array.three bytes.

First byte is port:

defined as following:

#define COM1 1
#define COM2 2
#define COM3 3
#define COM4 4
#define COM5 5
#define COM6 6
#define COM7 7
#define COM8 8
#define COM9 9

Second is the address of the reader.

When using broadcasting address 0xFF as ComAdr to call the function, the address of the reader will be writed back to parameter ComAdr;

When using a designated address 0x00~0xFE as ComAdr to call the function, the function will detect whether a specified address reader is connected to the designaged communication port.

Third value set the baud rate of the serial communication control.

baudrate	Actual baud rate
0	9600bps
1	19200 bps
2	38400 bps
4	56000 bps
5	57600 bps
6	115200 bps

Returns:

Return an array when successfully, non-zero value with first byte when error occurred.

Second is reader address,and last byte is a handle of reader.

3.1.3) CloseComPort():

Function description:

This function is used to disconnect the reader and release the corresponding communication port resources. In some development environment, the communication port resources must be released before exiting. Otherwise the operation system will become unstable.

Usage:

int ClosesComPort();

Parameter: None.

Returns:
Zero value when successfully, non-zero value when error occurred.

3.1.4) CloseSpecComPort():

Function description:
This function is used to disconnect the reader with the designated communication port and release the corresponding resources.
Usage:
int CloseSpecComPort(int Frmhandle);
Parameter:
FrmHandle: Handle of the corresponding communication port the reader is connected. The handle value is got when calling function AutoOpenComPort or OpenComPort.
Returns:
Zero value when successfully, non-zero value when error occurred.

3.1.5)GetReaderInformation():

Function description:
This function is used to get reader-related information such as reader address(ComAdr), firmware version, supported protocol type and InventoryScanTime.
Usage:
int[] GetReaderInformation(int[]arr);
Parameter:
arr: Input array.three bytes.
First byte is the address of the reader
Second is the handle of port.
Returns:
Return an array when successfully, non-zero value with first byte when error occurred.
Second to third is firmware version information, Fourth is reader type , Fifth is supported protocol information, Sixth is frequency, the specific definitions, see the user manual , Seventh is the specific definitions, see the user manual, Eighth is power of reader, and last byte is time limit for inventory command.

3.1.6) WriteComAdr ():

Function description:
This function is used to set a new address of the reader. The address value will store in reader's inner nonvolatile memory. Default address value is 0x00. The value range is 0x00~0xFE. The address 0xFF is reserved as the broadcasting address.When user try to write a 0xFF to ComAdr, the reader will set the value to 0x00 automatically.
Usage:
int WriteComAdr(int[] arr);
Parameter:

arr: Input array.three bytes.
First byte is the address of the reader
Second is the he new address of the reader.
Third is the handle of the port
Returns:
Zero value when successfully, non-zero value when error occurred.

3.1.7) WriteScanTime ():

Function description:
This function is used to set a new value to InventoryScanTime of an appointed reader. The range is 3~255 corresponding to 3*100ms~255*100ms InventoryScanTime. The default value of InventoryScanTime is 30*100ms.
Usage:
int WriteScanTime(int[] arr);
Parameter:
arr: Input array.three bytes.
First byte is the address of the reader
Second is the value of InventoryScanTime.
Third is the handle of the port
Returns:
Zero value when successfully, non-zero value when error occurred.

3.1.8) SetPowerDbm ():

Function description:
The function is used to set the power of reader.
Usage:
int SetPowerDbm(int[] arr);
Parameter:
arr: Input array.three bytes.
First byte is the address of the reader
Second is the power of reader.
Third is the handle of the port.
Returns:
Zero value when successfully, non-zero value when error occurred.

3.1.9) Writedfre ():

Function description:
The function is used to set the reader working of the lower limit and the upper limit of frequency.
Usage:
int Writedfre(int[] arr);
Parameter:
arr: Input array.four bytes.
First byte is the address of the reader
Second and third is the specific definitions, see the user manual

Fourth is the handle of the port.

Returns:

Zero value when successfully, non-zero value when error occurred.

3.1.10) Writebaud ():

Function description:

The function is used to change the serial port baud rate.

Usage:

int Writebaud(int[] arr);

Parameter:

arr: Input array.three bytes.

First byte is the address of the reader

Second is the baud rate of reader is 57600bps. Range is 0~6.

baudrate	Actual baud rate
0	9600bps
1	19200 bps
2	38400 bps
4	56000 bps
5	57600 bps
6	115200 bps

Reader support 43000bps baud rate, but ApdComPort control in DLL is not support 43000bps.

Third is the handle of the port.

Returns:

Zero value when successfully, non-zero value when error occurred.

3.2) EPCC1-G2 Function:

3.2.1) Inventory_G2 ():

Function description:

The function is used to detect tags in the inductive area and get their EPC values.

Usage:

int[] Inventory_G2(int[]arr);

Parameter:

arr: Input array.two bytes.

First byte is the address of the reader

Second is the handle of port.

Returns:

Return an array when successfully, first byte value:

0x01 Return before Inventory finished

0x02 the Inventory-scan-time overflow

0x03 More Data
0x04 Reader module MCU is Full
others when error occurred.
Second byte is tag number.
Third byte is EPC length and EPC's byte count.
Other bytes is EPC length +EPC.

3.2.2) ReadCard_G2 ():

Function description:

The function is used to read part or all of a Tag's Password, EPC, TID, or User memory. To the word as a unit, start to read data from the designated address.

Usage:

int[] ReadCard_G2(int[]arr);

Parameter:

arr: Input array.
First byte is the address of the reader
Second is the byte length of EPC.
Third to EPC-length+2 is EPC number.
EPC-length+3th is select the memory area to read.
0x00: Password area;
0x01: EPC memory area;
0x02: TID memory area;
0x03: User's memory area;
Other value when error occurred.
EPC-length+4th is the address of tag data to read (Word/Hex). Such as, 0x00 stand in start to read data from first word, 0x01 stand in start to read data from second word, and so on.
EPC-length+5th is the number of word to read. Can not set 0 or 120, otherwise, return the parameter error information. Num <= 120
EPC-length+6th to EPC-length+9th is Password.
EPC-length+10th is EPC masking starting address of byte.
EPC-length+11th is Masking bytes.
EPC-length+12th is EPC masking Flag.
0x00:disabled;
0x01:enabled;
EPC-length+13th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is the Read data.

3.2.3) WriteCard_G2 ():

Function description:

The function is used to write several words in a Tag's Reserved, EPC, TID, or User memory.

Usage:

int[] WriteCard_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the byte length of EPC.

Third to EPC-length+2 is EPC number.

EPC-length+3th is select the memory area to read.

0x00: Password area;

0x01: EPC memory area;

0x02: TID memory area;

0x03: User's memory area;

Other value when error occurred.

EPC-length+4th is the address of tag data to write (Word/Hex).Such as,0x00 stand in start to read data from first word, 0x01 stand in start to read data from second word, and so on.

EPC-length+5th is the byte number of word to write.

EPC-length+6th to EPC-length+ Writedatalen +5th is the array of the word to be written. For example,WordPtr equal 0x02, then the first word in Data write in the address 0x02 of designated Mem, the second word write in 0x03, and so on.

EPClength+Writedatalen+6th to EPClength+Writedatalen+9 is tag's password.

EPClength+Writedatalen+10th is EPC masking starting address of byte.

EPClength+Writedatalen+11th is Masking bytes.

EPClength+Writedatalen+12th EPC masking Flag.

0x00:disabled;

0x01:enabled;

EPClength+Writedatalen+13th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.4) EraseCard_G2 ():

Function description:

The function is used to erase multiple words in a Tag's Password, EPC, TID, or User memory.

Usage:

int[] EraseCard_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the byte length of EPC.

Third to EPC-length+2 is EPC number.

EPC-length+3th is select the memory area to read.

0x00: Password area;

0x01: EPC memory area;

0x02: TID memory area;

0x03: User's memory area;

Other value when error occurred.

EPC-length+4th is the address of tag data to erase (Word/Hex). Such as, 0x00 stand in start to

read data from first word, 0x01 stand in start to read data from second word, and so on.
EPC-length+5th is the byte number of word to erase.
EPClength +6th to EPClength +9 is tag's password.
EPClength +10th is EPC masking starting address of byte.
EPClength +11th is Masking bytes.
EPClength +12th EPC masking Flag.
0x00:disabled;
0x01:enabled;
EPClength +13th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.5) SetCardProtect_G2 ():

Function description:

The function is used to set Password area as readable and writeable from any state, readable and writeable from the secured state, permanently readable and writeable, never readable and writeable.It used to set EPC, TID or User as writeable from any state, writeable from the secured state, permanently writeable, never writeable.

Usage:

Int [] SetCardProtect_G2(int[]arr);

Parameter:

arr: Input array.
First byte is the address of the reader
Second is the byte length of EPC.
Third to EPC-length+2 is EPC number.
EPC-length+3th is select the memory area to read.
0x00: Password area;
0x01: EPC memory area;
0x02: TID memory area;
0x03: User's memory area;
Other value when error occurred.
EPC-length+4th is Setprotect

When Select is 0x00 or 0x01, SetProtect means as follows:

0x00: readable and writeable from any state.
0x01: permanently readable and writeable.
0x02: readable and writeable from the secured state.
0x03: never readable and writeable

When Select is 0x02, 0x03 or 0x04, SetProtect means as follows:

0x00: writeable from any state.
0x01: permanently writeable.
0x02: writeable from the secured state.
0x03: never writeable.

EPClength +5th to EPClength +8 is tag's password.
EPClength +9 is EPC masking starting address of byte.

EPClength +10 is Masking bytes.
EPClength +11 is EPC masking Flag.
0x00:disabled;
0x01:enabled;
EPClength +12th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.6) DestroyCard_G2 ():

Function description:

The function is used to destroy tag. After the tag destroyed, it never process command.

Usage:

int[] DestroyCard_G2(int[]arr);

Parameter:

arr: Input array.
First byte is the address of the reader
Second is the byte length of EPC.
Third to EPC-length+2 is EPC number.
EPC-length+3th to EPC-length+6th is password.
EPClength +7th is EPC masking starting address of byte.
EPClength +8th is Masking bytes.
EPClength +9th is EPC masking Flag.
0x00:disabled;
0x01:enabled;
EPClength +10th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.7) WriteEPC_G2 ():

Function description:

The function is used to write EPC value in a Tag's EPC memory. Random write one tag in the antenna.

Usage:

int[] WriteEPC_G2(int[]arr);

Parameter:

arr: Input array.
First byte is the address of the reader
Second to fifth is the password.
Sixth is EPC length.
Seventh to EPC-Length+6th is EPC to write. .
EPC-Length+7th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.8) SetReadProtect_G2 ():

Function description:

The function is used to set designated tag read protection. After the tag destroyed, it never process command. Even if inventory tag, reader can not get the EPC number.Only NXP's UCODE EPC G2X tags valid.

Usage:

int[] SetReadProtect_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the byte length of EPC.

Third to EPC-length+2 is EPC number.

EPC-length+3th to EPC-length+6th is password.

EPClength +7th is EPC masking starting address of byte.

EPClength +8th is Masking bytes.

EPClength +9th is EPC masking Flag.

0x00:disabled;

0x01:enabled;

EPClength +10th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.9) SetMultiReadProtect_G2 ():

Function description:

The function is used to random set one tag read protection in the antenna.The tag must be have the same access password.Only NXP's UCODE EPC G2X tags valid.

Usage:

int[] SetMultiReadProtect_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second to Fifth is the password.

Sixth is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.10) RemoveReadProtect_G2 ():

Function description:

The function is used to reset only one tag read protection in the antenna.The tag must be have the same access password.Only NXP's UCODE EPC G2X tags valid.

Usage:

int[] RemoveReadProtect_G2(int[]arr);

Parameter:
arr: Input array.
First byte is the address of the reader
Second to Fifth is the password.
Sixth is the handle of port.

Returns:
Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none.

3.2.11) CheckReadProtected_G2 ():

Function description:
The function is used to check only one tag in the antenna whether the tag is protected. It can not chek the tag whether the tag support protection setting.Only NXP's UCODE EPC G2X tags valid.
Usage:
int[] CheckReadProtected_G2(int[]arr);
Parameter:
arr: Input array.
First byte is the address of the reader
Second the handle of port.
Returns:
Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is Readpro. 0x00 Tag is unprotected0x01 Tag is protected.

3.2.12) SetEASAlarm_G2 ():

Function description:
The function is used to set or reset the EAS status bit of designated tag. Only NXP's UCODE EPC G2X tags valid.
Usage:
int[] SetEASAlarm_G2(int[]arr);
Parameter:
arr: Input array.
First byte is the address of the reader
Second is the byte length of EPC.
Third to EPC-length+2 is EPC number.
EPC-length+3th to EPC-length+6th is password.
EPClength +7th is EPC masking starting address of byte.
EPClength +8th is Masking bytes.
EPClength +9th is EPC masking Flag.
0x00:disabled;
0x01:enabled;
EPClength +10th is EAS. The bit0 of EAS is 0, it means setting EAS closed.
The bit0 of EAS is 1, it means setting EAS opened.bit1 ~ bit7 default is 0.

EPClength +11th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none

3.2.13) CheckEASAlarm_G2 ():

Function description:

The function is used to check EAS status bit of any tag in the antenna. Only NXP's UCODE EPC G2X tags valid.

Usage:

int[] CheckEASAlarm_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred,Then second byte is error code. First is Zero value then EAS,0xFB no EAS .then other byte is none

3.2.14) LockUserBlock_G2 ():

Function description:

The function is used to permanently lock the designated data in designated tag's user memory. The locked data can be read only, but not written and not erased. Only NXP's UCODE EPC G2X tags valid.

Usage:

int[] LockUserBlock_G2(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the byte length of EPC.

Third to EPC-length+2 is EPC number.

EPC-length+3th to EPC-length+6th is password.

EPClength +7th is EPC masking starting address of byte.

EPClength +8th is Masking bytes.

EPClength +9th is EPC masking Flag.

0x00:disabled;

0x01:enabled;

EPClength +10th is the address to lock:

0 or 1	permanently lock block data 0 and 1
2 or 3	permanently lock block data 2 and 3
4 or 5	permanently lock block data 4 and 5
6 or 7	permanently lock block data 6 and 7

8 or 9 permanently lock block data 8 and 9
10 or11 permanently lock block data 10 and 11
12 or13 permanently lock block data 12 and 13

EPClength +11th is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Then second byte is error code. Zero value when successfully,then other byte is none

3.3) 18000-6B Function:

3.3.1) Inventory_6B ():

Function description:

The function is used to detect only one tag in the inductive area and get their ID values.If more than one tag in the inductive area at the same time, reader may be detect nothing.

Usage:

int[] Inventory_6B(int[]arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Zero value when successfully,then 2-9 byte is tag's ID.

3.3.2) Inventory2_6B ():

Function description:

The function is used to according to the given conditions detect tags in the inductive area and get their ID values.

Usage:

Int[] Inventory2_6B (int[] arr);

Parameter:

arr: Input array.

First byte is the address of the reader

Second is is the the condition of detecting tags.

0x00: equal to condition.

0x01: unequal to condition.

0x02: greater than to condition.

0x03: less than to condition.

Third is the tag's start address to compare.

Fourth is the data used to compare. Highest bit in the mask correspond with the far-left byte in the ConditionContent.The corresponding bit in the mask is 1 to compare the bit in the ConditionContent with the corresponding byte in the tag.The corresponding bit in the mask is 0, not compare.

Fifth to Twelfth is the array used to compare

Thirteenth is the handle of port.

Returns:

Return an array .first value:

0x15 Return before Inventory finished

0x16 the Inventory-scan-time overflow

0x17 More Data

0x18 Reader module MCU is Full

others when error occurred.

Second is tag's number.

Other bytes are tag's ID.

3.3.3) ReadCard_6B ():

Function description:

The function is used to start to read several bytes from the designated address.

Usage:

int[] ReadCard_6B(int[]arr);

Parameter:

First byte is the address of the reader

Second to Ninth is ID.

Tenth is the start address to read data. Range is 8~223. If the adrress more than range, reader will return parameter error information.

Eleventh is the number of bytes to read, range is 1~32. If StartAddress + Num greater than 224, or Num greater than 32 or is zero, reader will return parameter error information.

Twelfth is the handle of port.

Returns:

Return an array ,first is non-zero value when error occurred.Zero value when successfully,then other bytes are data read from tag .

3.3.4) WriteCard_6B ():

Function description:

The function is used to start to write several bytes from the designated address.

Usage:

int[] WriteCard_6B(int[]arr);

Parameter:

First byte is the address of the reader

Second to Ninth is ID.

Tenth is the start address to write data. Range is 8~223. If the adrress more than range, reader will return parameter error information.

Eleventh is the number of bytes to write, range is 1~32. If StartAddress + Num greater than 224, or Num greater than 32 or is zero, reader will return parameter error information.

Writedatalen+11th is the number of bytes written successfully start from the high byte in the Writedata..

Writedatalen+12th is the handle of port

Returns:

Return an array, first is non-zero value when error occurred.Zero value when

successfully,second byte is write word bumber,third byte is error code.

3.3.5) CheckLock_6B ():

Function description:

The function is used to check whether the designated byte is locked.

Usage:

int[] CheckLock_6B(int[]arr);

Parameter:

First byte is the address of the reader

Second to Ninth is ID.

Tenth is the address is used to check whether is locked. Range is 0~223. Beyond this range, reader will return parameter error.

Eleventh is the handle of port.

Returns:

Return an array. first is non-zero value when error occurred.Zero value when successfully,second byte is lock state, 0x00, the byte is unlocked.0x01, the byte is locked, cannot lock it again third byte is error code.

3.3.6) LockByte_6B ():

Function description:

The function is used to lock the designated byte.

Usage:

int[] LockByte_6B(int[]arr);

Parameter:

First byte is the address of the reader

Second to Ninth is ID.

Tenth is the address to lock. Range is 8~223. Beyond this range, reader will return parameter error.

Eleventh is the handle of port.

Returns:

Return an array. first is non-zero value when error occurred.Zero value when successfully,second byte is error code..

4. Return Value Definition

#define InventoryReturnEarly_G2 0x01 Return before Inventory finished

#define InventoryTimeOut_G2 0x02 the Inventory-scan-time overflow

#define InventoryMoreData_G2 0x03 More Data

#define ReadermoduleMCUFull_G2 0x04 Reader module MCU is Full

#define AccessPasswordError 0x05 Access password error

#define DestroyPasswordError 0x09 Destroy password error

#define DestroyPasswordCannotZero 0x0a Destroy password error cann't be Zero

#define TagNotSupportCMD 0x0b Tag Not Support the command

#define AccessPasswordCannotZero 0x0c Use the commmand,Access Password Cann't be Zero

#define TagProtectedCannotSetAgain 0x0d Tag is protected,cannot set it again

#define TagNoProtectedDonotNeedUnlock 0x0e Tag is unprotected,no need to reset it

#define ByteLockedWriteFail 0x10 There is some locked bytes,write fail

#define CannotLock 0x11 can not lock it

#define LockedCannotLockAgain 0x12 is locked,cannot lock it again

#define ParameterSaveFailCanUseBeforeNoPower

0x13 Save Fail,Can Use Before Power

#define CannotAdjust 0x14 Cannot adjust

#define InventoryReturnEarly_6B 0x15 Return before Inventory finished

#define InventoryTimeOut_6B 0x16 Inventory-Scan-Time overflow

#define InventoryMoreData_6B 0x17 More Data

#define ReadermoduleMCUFull_6B 0x18 Reader module MCU is full

#define NotSupportCMDOrAccessPasswordCannotZero

0x19 Not Support Command Or AccessPassword Cannot be Zero

#define CMDExecuteErr 0xF9 Command execute error

#define GetTagPoorCommunicationCannotOperation

0xFA Get Tag,Poor Communication,Inoperable

#define NoTagOperation 0xFB No Tag Operable

#define TagReturnErrorCode 0xFC Tag Return ErrorCode

#define CMDLengthWrong 0xFD Command length wrong

#define IllegalCMD 0xFE Illegal command

#define ParameterError 0xFF Parameter Error

#define CommunicationErr 0x30 Communication error

#define RetCRCErr 0x31 CRC checksummat error

#define RetDataErr 0x32 Return data length error

#define CommunicationBusy 0x33 Communication busy

#define ExecuteCmdBusy 0x34 Busy,command is being executed

#define ComPortOpened 0x35 ComPort Opened

#define ComPortClose 0x36 ComPort Closed

#define InvalidHandle 0x37 Invalid Handle

#define InvalidPort 0x38 Invalid Port

#define RecmdErr 0XEE Return command error

5.ErrorCode Definition

#define OtherError 0x00 Other error
#define MemoryOutPcNotSupport 0x03 Memory out or pc not support
#define MemoryLocked 0x04 Memory Locked and unwritable
#define NoPower 0x0b No Power,memory write operation cannot be executed
#define NotSpecialError 0x0f Not Special Error,tag not support special errorcode